Instituto Superior Técnico

BSc Aerospace Engineering

Multivariable, Nonlinear and Optimal Control

Final Report: Linear State Feedback Control of an Inverted Pendulum

Professor: João Manuel Lage de Miranda Lemos

Laboratory Professor: Carlos Filipe Gomes Bispo

Grup: 35

Lucas Moreira Nº 109657

Tomás Teixeira Nº 109766

Pedro Machado Nº 110718

Abril, 2026

Contents

I Introduction 2
- I-A Context and Objectives . . . 2
- I-B System Description . . . 2

II Theoretical Analysis and Control Design 3
- II-A Space-State Formulation . . . 3
- II-B State-Space Analysis . . . 3
  - II-B1 Q1 - System Poles . . . 3
  - II-B2 Q2 - Controllability . . . 3
  - II-B3 Q3 - Observability . . . 4
  - II-B4 Q4 - Bode Plots . . . 4
- II-C LQR Controller Design . . . 5
- II-D State Estimator Design - LQE . . . 6
- II-E Baseline LQG Simulation . . . 7

III Laboratory Work and Experimental Results 9
- III-A Initial Hardware Testing . . . 9
  - III-A1 LQE Validation . . . 9
  - III-A2 LQG Testing . . . 9
  - III-A3 Analysis and Problems . . . 10
- III-B LQG Integrator . . . 10
- III-C LQG Integrator + Deadzone Compensator . . . 10

IV Conclusion 12

V Anexos 13

I. Introduction

A. Context and Objectives

This report details the work developed within the scope of the Multivariable, Non-linear, and Optimal Control course laboratory project. The primary objective is to design, simulate, and implement a robust control strategy to stabilize a highly non-linear and inherently unstable system using a pre-existing experimental setup of a rotary inverted pendulum.

The adopted methodology relies on linearizing the plant's mathematical model around its unstable equilibrium point (the upward vertical position) to derive a Linear Time-Invariant (LTI) state-space representation. Based on this model, a Linear Quadratic Gaussian (LQG) control architecture is developed. This comprises a Linear Quadratic Regulator (LQR) to compute the optimal state feedback law, and a Linear Quadratic Estimator (LQE, or Kalman Filter) to estimate the system's full internal state from noisy measurements. The project covers the theoretical analysis of the system, validation through MATLAB/Simulink simulations, and practical real-time implementation on the laboratory's physical hardware, specifically addressing real-world non-linearities such as motor deadzones and steady-state errors.

B. System Description

The physical plant utilized in this work is a rotary inverted pendulum apparatus (a Quanser experimental setup). The mechanical structure consists of a motorized rotary arm that actively maneuvers an unactuated, free-swinging pendulum rod attached to its extremity.

The system operates as a Single-Input Multiple-Output (SIMO) process. The single control input u is the continuous voltage applied to the DC motor that drives the horizontal rotary arm. To monitor the system's behavior, two physical sensors provide the measured output vector y:

- A potentiometer measuring the angular position of the rotary arm α.
- An optical encoder measuring the precise angular position of the pendulum relative to the arm β.

Because the system's complete state vector includes both the angular positions α, β and their respective angular velocities $\dot{\alpha}, \dot{\beta}$, but only the positions are directly measured, the system requires a state observer. Furthermore, the provided physical setup interfaces with the Simulink control environment via A/D and D/A data acquisition boards, allowing the designed controllers to be executed and tested on the real system in real-time.

II. Theoretical Analysis and Control Design

A. Space-State Formulation

The non-linear dynamics of the system imply that it needs to be linearized around an equilibrium point, which, given the objective of this report, is the unstable equilibrium where the pendulum is in the upright vertical position. This equilibrium corresponds to all state variables being equal to zero. The derivation of the non-linear model and the subsequent linearization are not the focus of this report and can be found in Chapter 2 of Samuel Balula's thesis **(reference needed)**. The linearized model of the system, which reasonably represents the dynamics, is given by:

$$\begin{cases} \dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u} \\ \mathbf{y} = C\mathbf{x} + D\mathbf{u} \end{cases}$$

where $\mathbf{x} = [\alpha\ \dot{\alpha}\ \beta\ \dot{\beta}\ i]^T$ is the 5-dimensional state vector, and i is the motor current. The one-dimensional input vector $\mathbf{u} = u$ corresponds to the DC voltage, in *Volts*, applied to the terminals of the DC motor through a power amplifier, directly controlling α. The output vector $\mathbf{y} = [\alpha\ \beta]^T$ consists of the sensor measurements, corresponding to the angular positions of the arm and the pendulum, respectively.

The system matrices A, B, C, and D are given by:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & -0.0174 & 20.7861 & -0.0023 & 57.5344 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & -0.0174 & 63.8319 & -0.0071 & 57.4388 \\ 0 & -232.0252 & 0 & 0 & -755.425 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 333.3 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

B. State-Space Analysis

In this section, we analyze the previously formulated state-space model in terms of system poles, controllability, observability, and frequency response.

1) Q1 - System Poles: The system poles are the eigenvalues of the dynamics matrix A. Using MATLAB's `eig()` function, the following poles were obtained:

TABLE I: Características dos Modos do Sistema

Polos	ω (rad/s)	τ (s)
0	0	∞
-737.3	7.37×10^2	1.36×10^{-3}
-19.36	1.94×10^1	5.16×10^{-2}
-5.761	5.76	1.74×10^{-1}
6.997	7.00	-1.43×10^{-1}

Analysis of the poles shows that the system has three stable modes: two correspond to the mechanical damping of the pendulum, $\lambda_3 = -19.3673$ and $\lambda_4 = -5.7612$, and a very fast mode associated with the motor current, $\lambda_2 = -737.3184$. Additionally, the state α generates a pole at the origin, $\lambda_1 = 0$, due to being an integrator. This implies that after a perturbation, α will accumulate error over time without diverging or converging, reflecting the fact that the arm rotates without friction and does not naturally settle to an equilibrium.

The single unstable mode, $\lambda_5 = 6.9974$, is related to the vertical pendulum position, which is the primary state we aim to control.

2) Q2 - Controllability: A system is controllable if it is possible to transition from any state x_1 to another arbitrary state x_2 by applying an input $u(t)$ over a finite time interval. Knowing whether the system is controllable is crucial for understanding the limitations of the controller.

The controllability matrix is defined as:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}, \tag{1}$$

where n is the number of states. Using MATLAB's `ctrb(A,B)` function, we obtained:

$$\mathcal{C} = \begin{bmatrix} 0 & 0 & 1.918\times10^4 & -1.449\times10^7 & 1.069\times10^{10} \\ 0 & 1.918\times10^4 & -1.449\times10^7 & 1.069\times10^{10} & -7.881\times10^{12} \\ 0 & 0 & 1.915\times10^4 & -1.446\times10^7 & 1.067\times10^{10} \\ 0 & 1.915\times10^4 & -1.446\times10^7 & 1.067\times10^{10} & -7.869\times10^{12} \\ 3.333\times10^2 & -2.518\times10^5 & 1.858\times10^8 & -1.370\times10^{11} & 1.010\times10^{14} \end{bmatrix}$$

The system is controllable if the rank of the controllability matrix equals the number of states. Using MATLAB's `rank()` function, we obtained $\text{rank}(\mathcal{C}) = 5$, which equals the number of states. Therefore, the system is controllable.

3) Q3 - Observability: A system is observable if, for any initial state $x(0)$, knowledge of the input $u(t)$ and output $y(t)$ over a finite time interval uniquely determines $x(0)$. Although the system is controllable, implementing a state-feedback controller requires knowledge of the full state[1]. Since the state is not directly measurable, it must be reconstructed from the outputs, which is only possible if the system is observable.

The observability matrix is defined as:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \tag{2}$$

Using MATLAB's `obsv()` function, the observability matrix was calculated as:

$$\mathcal{O} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & -1.740\times10^{-2} & 2.079\times10^1 & -2.316\times10^{-3} & 5.753\times10^1 \\ 0 & -1.738\times10^{-2} & 6.383\times10^1 & -7.114\times10^{-3} & 5.744\times10^1 \\ 0 & -1.335\times10^4 & -5.096\times10^{-1} & 2.079\times10^1 & -4.346\times10^4 \\ 0 & -1.333\times10^4 & -8.152\times10^{-1} & 6.383\times10^1 & -4.339\times10^4 \\ 0 & 1.008\times10^7 & -2.762\times10^5 & 3.027\times10^1 & 3.207\times10^7 \\ 0 & 1.007\times10^7 & -2.729\times10^5 & 2.960\times10^1 & 3.202\times10^7 \end{bmatrix} \tag{3}$$

Applying the `rank()` function shows that $\text{rank}(\mathcal{O}) = 5$, which equals the number of states. Therefore, the system is observable.

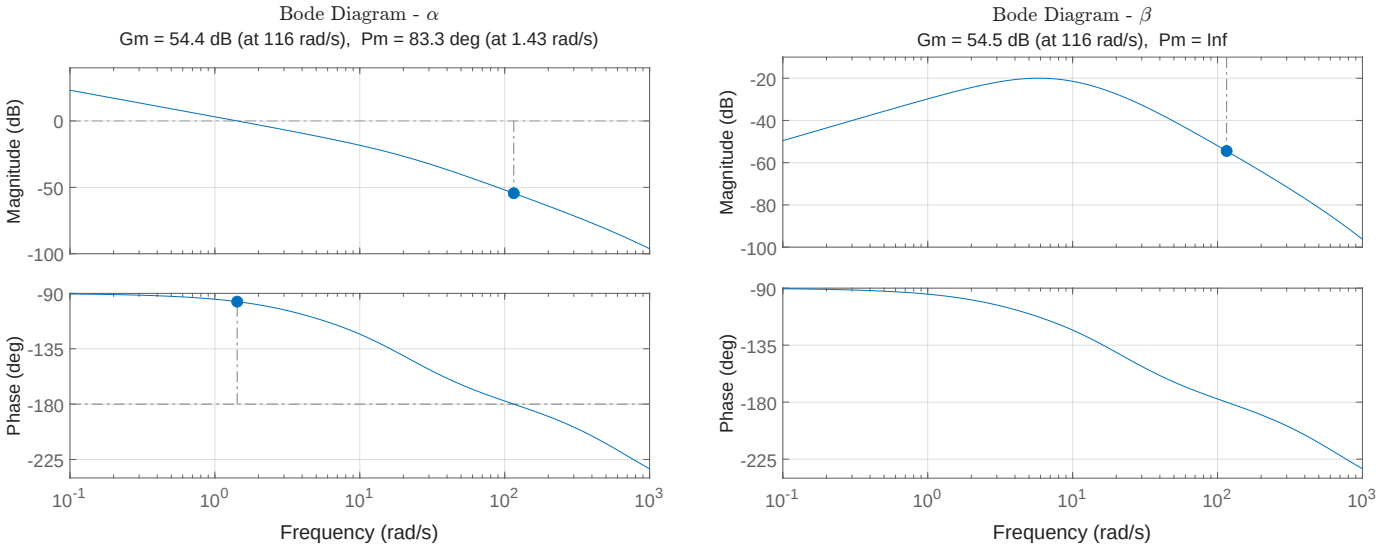
4) Q4 - Bode Plots: To evaluate the frequency response of the system, the transfer functions of α and β relative to the input u were determined, resorting to the function `ss2tf()`:

$$\frac{\alpha(s)}{u(s)} = \frac{1.92\times10^4\,s^2 + 92.07\,s - 8.26\times10^5}{s^5 + 755.45\,s^4 + 1.33\times10^4\,s^3 - 4.82\times10^4\,s^2 - 5.76\times10^5\,s} \tag{4}$$

$$\frac{\beta(s)}{u(s)} = \frac{0.1915\,s^2}{s^5 + 755.45\,s^4 + 1.33\times10^4\,s^3 - 4.82\times10^4\,s^2 - 5.76\times10^5\,s} \tag{5}$$

With the above transfer functions, we generated the Bode plots to characterize their frequency response.

[1]State-feedback control requires knowledge of the full state since the control law $u(t) = -Kx(t)$ explicitly depends on the state vector. In this project, an estimator was designed to reconstruct the state, resulting in the control law $u(t) = -K\hat{x}(t)$. State estimation is only possible if the system is observable, making observability a prerequisite for implementing state-feedback control when the state is not directly measurable.



(a) Bode plot for $\frac{\alpha(s)}{u(s)}$ (b) Bode plot for $\frac{\beta(s)}{u(s)}$

Figura 1: Open-loop Bode plots of the system evaluating both outputs.

The frequency response of $\frac{\alpha(s)}{u(s)}$ is characterized by a large gain margin, $G_m = 54.4$ dB, and a large phase margin, $P_m = 83.3°$. For frequencies below 1 rad/s, $\frac{\alpha(s)}{u(s)}$ amplifies inputs, whereas for higher frequencies this transfer function attenuates inputs. The transfer function $\frac{\beta(s)}{u(s)}$ is also characterized by a large gain margin, $G_m = 54.4$ dB, but possesses an infinite phase margin because the magnitude curve never crosses the 0 dB threshold.

C. LQR Controller Design

The initial approach to controlling the system involved designing a simple Linear Quadratic Regulator (LQR), implementing the state-feedback control law:

$$u(t) = -Kx(t) \tag{6}$$

The associated closed-loop state-space model becomes $\dot{x} = (A - BK)x$. Since the pair (A, B) is controllable, it is theoretically possible to place the poles of the system in any arbitrary position. To calculate the optimal gain matrix K, it is necessary to define a quadratic cost function J:

$$J = \frac{1}{2}\int_0^\infty \left(\mathbf{x}^T Q\mathbf{x} + \mathbf{u}^T R\mathbf{u}\right) dt \tag{7}$$

Here, Q is the state weighting matrix that penalizes deviations from the desired state, and R is the control weighting matrix that penalizes actuator effort. By tuning the ratio between Q and R, we balance aggressive trajectory tracking against control energy consumption.

The optimal state-feedback gain matrix K that minimizes this cost function is given by the expression $K = R^{-1}B^T P$. To obtain the matrix P, the Continuous Algebraic Riccati Equation (CARE) must be solved:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0 \tag{8}$$

In MATLAB, this process is handled automatically by the `lqr()` function. Based on this state-space model and the resulting optimal gain, we designed a preliminary Simulink model to simulate the plant and controller.

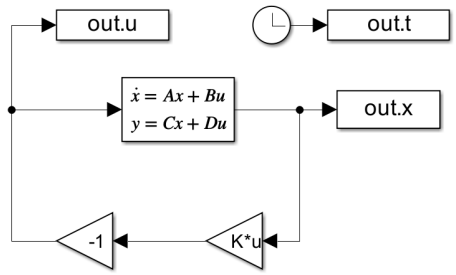


Figura 2: Simulink implementation of the LQR controlled system.

By running the simulink for the initial condition $x_0 = \begin{bmatrix} 0.1 & 0 & 0 & 0 & 0 \end{bmatrix}$, it is possible to observe the evolution of the five states, converging to an equilibrium.

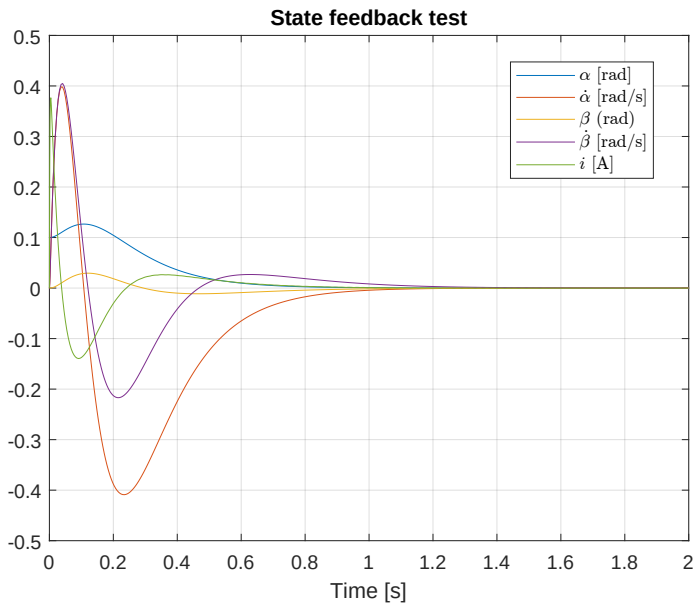


Figura 3: Time-domain response of the LQR controller

D. State Estimator Design - LQE

Because the observed output only consists of α and β, it is necessary to design an estimator (Kalman Filter) that computes the entire state vector from the limited observed states. Since the system is observable, it is guaranteed that the state vector can be reconstructed by observing outputs α and β over an interval of time.

The estimated state model is given by:

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}) \tag{9}$$

The observer adds its own dynamics to the overall system, governed by the eigenvalues of $A - LC$. To compute the optimal estimator gain L, we consider the process noise covariance matrix Q_r and the measurement noise covariance matrix R_r. The ratio of Q_r to R_r indicates how much the estimator trusts the physical measurements versus the mathematical model's predictions. High values in Q_r paired with low values in R_r indicate that the system should trust the incoming sensor measurements more.

The estimator gain L is obtained via the expression $L = PC^T R_r^{-1}$, where the matrix P (the error covariance) is found by solving the Continuous Algebraic Riccati Equation (CARE):

$$AP + PA^T - PC^T R_r^{-1} CP + GQ_r G^T = 0 \tag{10}$$

In the present work, we computed the optimal estimator gain by resorting to MATLAB's `lqe()` function.

E. Baseline LQG Simulation

By combining the LQR controller with the LQE observer, we create a Linear Quadratic Gaussian (LQG) controller. In this architecture, the estimated states are fed to the control law, which then computes the inputs to the plant. By combining Equation 7 with Equation 9, it is possible to arrive at the unified model for the controller block:

$$\dot{\hat{x}} = (A - BK - LC)\hat{x} + Ly, \tag{11}$$

$$u(t) = -K\hat{x}(t). \tag{12}$$

The implementation of the observer paired with the controller—featuring the dynamic matrix $A - BK - LC$, input matrix L, and output matrix $-K$—led us to the following Simulink block diagram implementation:

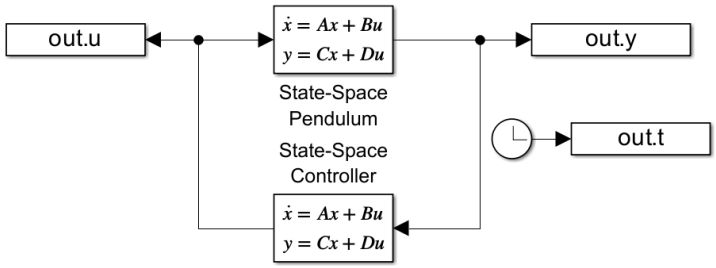
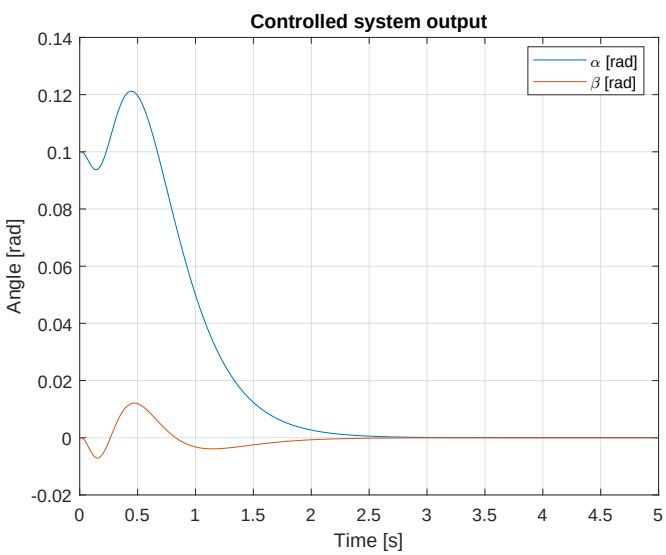
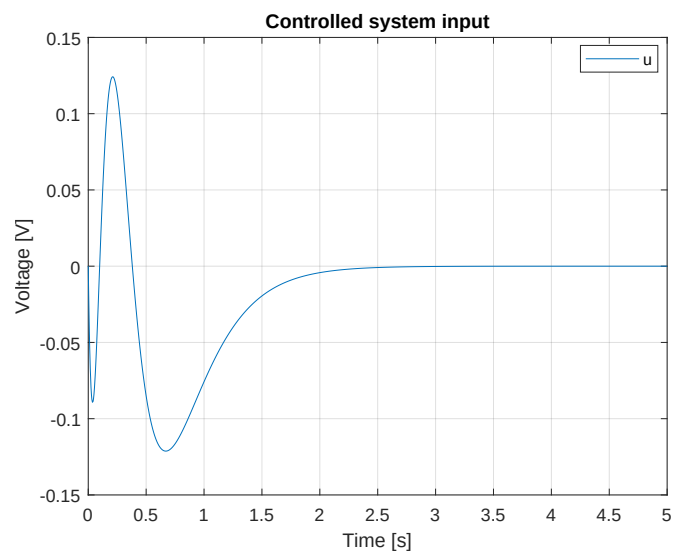


Figura 4: Simulink implementation of the complete LQG system (Estimator + Controller).

By running the simulink with the initial condition $x_0 = \begin{bmatrix} 0.1 & 0 & 0 & 0 & 0 \end{bmatrix}$, it is possible to obtain the evolution of the α and β states, as well as the voltage u outputed to the plant.



(a) Evolution of states α and β

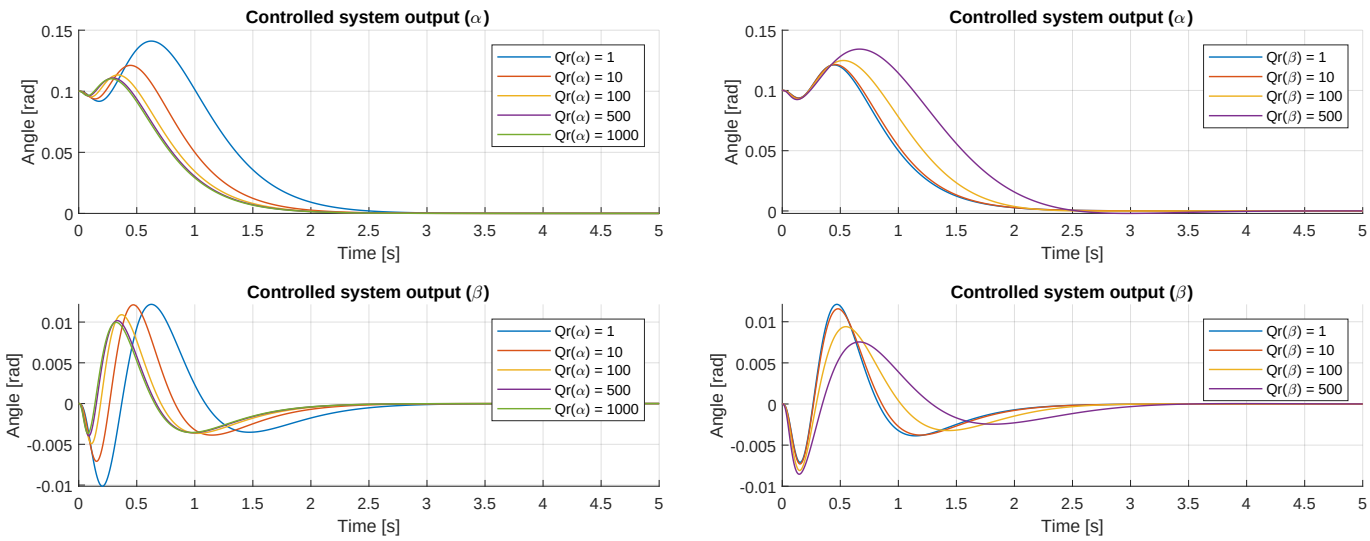


(b) Evolution of the output u

It is possible to observe that both states are tracking the reference, with some minor overshoot. At this stage, it became paramount to understand the influence Q_r and R_r had on the response of the system. It is noted that multiplying Q_r and R_r by a constant K does not influence the system response. This is because any multiplying constant in the cost function (7) can be factored out of the integral and does not change the optimal gain K. Consequently, only the relative weighting between Q_r and R_r determines the trade-off between minimizing state deviations and minimizing control effort.

Therefore, we varied the values of Q_{r1} and Q_{r3}[2] with a fixed R_r and vice-versa, in order to understand their effect on the system.

[2]To improve readibility, we adopted the convention of refering to entries of the Q_r matrix based on the state they influence. For example $Q_{r1} = Q_r(\alpha)$ and $Q_{r3} = Q_r(\beta)$



(a) Evolution of states α and β

(b) Evolution of the output u

Figura 6: Effect of Q_r on the evolution of states α and β

From Figures 6a and 6b, it can be observed that increasing the weighting associated with $Q_r(\alpha)$ leads to a reduction in both the overshoot and settling time of the states α and β. This indicates that placing a higher penalty on deviations of α results in a faster and more well-damped overall system response.

Conversely, increasing the weighting $Q_r(\beta)$ produces a different effect: while it reduces the overshoot of state β, it tends to increase both the overshoot and settling time of state α. This highlights the coupling between the system states and illustrates the trade-off inherent in selecting the weighting matrices.

We also analyzed the influence of the control weighting matrix R_r while keeping Q_r fixed.

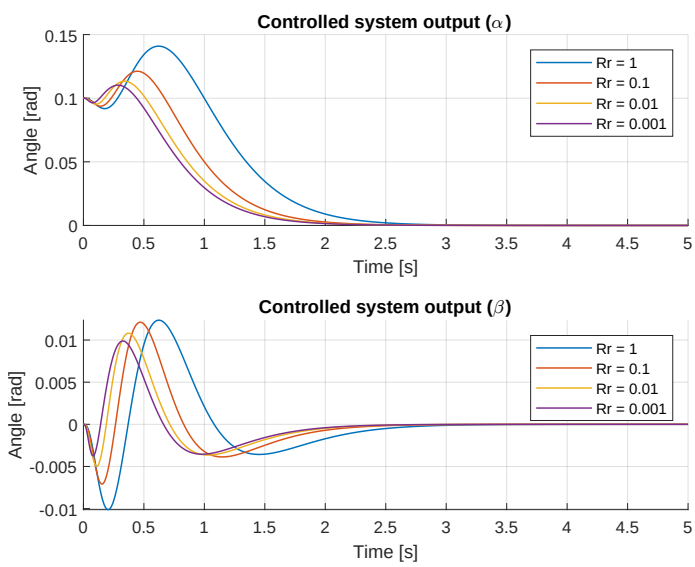


Figura 7: Influence of R_r on the system response.

From Figure 7, it is evident that decreasing R_r reduces both the overshoot and the settling time. This occurs because a smaller R_r penalizes the control effort less, allowing the controller to act more aggressively and respond faster to deviations from the reference.

III. Laboratory Work and Experimental Results

A. Initial Hardware Testing

1) LQE Validation: The first stage of the experimental procedure involved the validation of the LQE. To facilitate this, the LQG controller structure was decoupled into its two primary components: the LQR regulator and the LQE observer. This separation was implemented in the Simulink model METER-MODELO by using independent blocks, as shown in the system diagram in Figure [X]. This configuration allowed for the isolation of the estimator, enabling the monitoring of the observer's performance.

The primary objective of this test was first, to ensure that the estimated states $\hat{\alpha}$ and $\hat{\beta}$ accurately tracked the raw angular measurements; and second, to verify that the remaining three unmeasured states behaved smoothly and remained strictly within expected physical boundaries, without exhibiting erratic fluctuations or excessive noise amplification. This objective was achieved by tuning the estimator's noise covariance matrices, Q_e and R_e.

A fundamental observation regarding the experimental setup was the high precision of the discrete sensors. The Quanser platform utilizes a potentiometer for the rotary arm and a high-resolution optical encoder for the pendulum angle. Due to their discrete nature, the continuous measurement noise was found to be practically negligible. However, to ensure numerical stability in the Kalman Filter algorithm and to reflect a high degree of confidence in the hardware readings, the measurement noise covariance matrix was set to:

$$R_e = I_2 \cdot (\text{deg2rad}(0.1))^2 \tag{13}$$

With R_e fixed to admit a standard deviation of only 0.1°, the Q_e matrix was tuned empirically through hardware trials. In the Kalman Filter framework, it is the relative ratio between these covariance matrices (Q_e/R_e) that dictates the observer's gain and dynamic behavior. A high ratio yields a highly reactive filter, whereas a low ratio produces a slower, heavily filtered response.

To find the optimal balance, ensuring accurate tracking while avoiding overfitting, three distinct tests were performed by scaling the process noise identity matrix keeping R_e constant: $Q_e = 0.1 \cdot I_5$, $Q_e = I_5$, and $Q_e = 10 \cdot I_5$.

As illustrated in Figure [X], the system's real-time plots were analyzed for each configuration. The results demonstrated that while lower values of Q_e introduced a noticeable lag, the configuration with [INSERIR QUAL FOI O MELHOR] provided the best overall performance. It allowed the estimates $\hat{\alpha}$ and $\hat{\beta}$ to converge rapidly to the physical measurements with high fidelity. Crucially, the plots also confirmed that the three unmeasured states exhibited coherent dynamic trends, remaining well within the expected operational limits during the pendulum's motion. This thoroughly validated LQE setup was then adopted for all subsequent closed-loop experiments.

2) LQG Testing: Following the validation of the LQE, the complete LQG controller was implemented on the hardware. The weighting matrices for the LQR regulator, Q_r and R_r, were selected based on the optimal values determined during the theoretical simulation phase. However, for this test, we decided to maintain the default values of .....

The explicit objective of this hardware test was to demonstrate a fundamental limitation of the standard LQG architecture: its inability to stabilize the rotary arm (α) at a complete standstill at the origin.

To evaluate this, a physical disturbance was introduced to the system during operation. As illustrated in the experimental plots in Figure 8, while the controller successfully kept the pendulum upright, the rotary arm failed to return to the zero position. Instead, α exhibited a persistent steady-state error, drifting and coming to a stop away from the reference point.

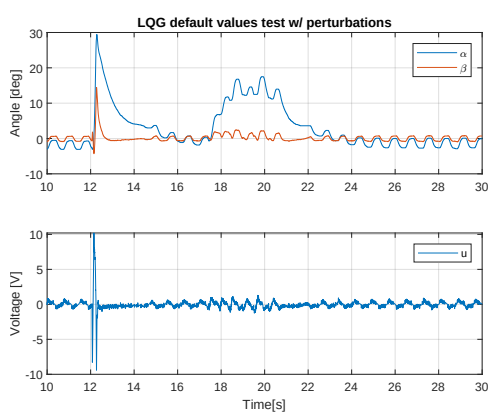


Figura 8: Default LQG response to a physical disturbance.

3) Analysis and Problems: An analysis of this drift revealed two coupled causes rooted in the hardware's physical properties and the controller's structure:

- Motor Deadzone: The DC motor only actuates when the applied voltage exceeds a specific threshold. Within this deadzone range, the control inputs (u) calculated by the LQR do absolutely nothing. As a consequence, the rotary arm will always exhibit a large amplitude of oscillation. The system drifts until the position error grows large enough for the controller to finally command a voltage that breaks through the deadzone, causing the motor to suddenly move, overshoot, and repeat the cycle.
- Absence of an Integrative State: The problem of the steady-state drift is fundamentally tied to the composition of our state vector. The standard LQR model relies strictly on the proportional feedback of the position and velocity states. Because it lacks an integrative state dedicated to accumulating the error of α over time, the controller has no "memory." It inherently accepts steady-state offsets whenever the proportional control effort generated by that offset is smaller than the deadzone resistance, causing the arm to settle away from the reference point.

B. LQG Integrator

C. LQG Integrator + Deadzone Compensator

Motor Deadzone Characterization and Compensation

To quantify the motor deadzone, a voltage ramp was applied to the actuator to identify the specific threshold required to initiate motion. Experimental results yielded a deadzone half-width of ± 0.34 [V]. It is noted that preliminary measurements indicated a half-width of approximately ± 0.20 [V]; however, these initial data points were not formally recorded. It is critical to acknowledge that this parameter is subject to stochastic variations influenced by environmental factors, such as operating temperature. For the subsequent simulations and the design of the deadzone compensator, the value of ± 0.20 [V] was adopted.

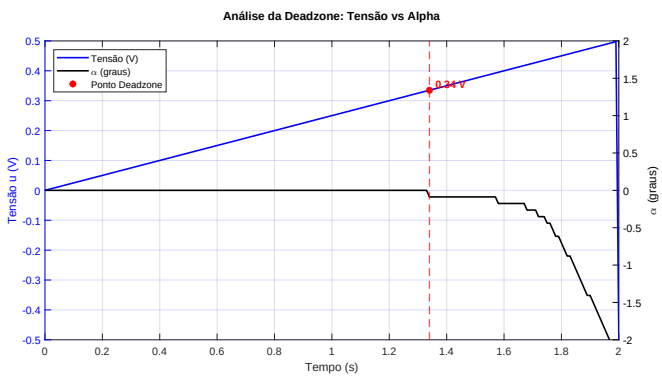


Figura 9: Motor deadzone measurement

The motor was modeled within the Simulink environment as a first-order system with a time constant $\tau = 0.01$ [s]. The model incorporates several non-linearities and constraints, including a voltage input saturation of ± 5 [V], output angular constraints of $\pm 45°$ for α and $\pm 15°$ for β, and a deadzone half-width of ± 0.20 [V]. A simulation was performed including the implemented α integrator to evaluate the uncompensated system response prior to the integration of the deadzone compensator. The resulting trajectory is illustrated below:

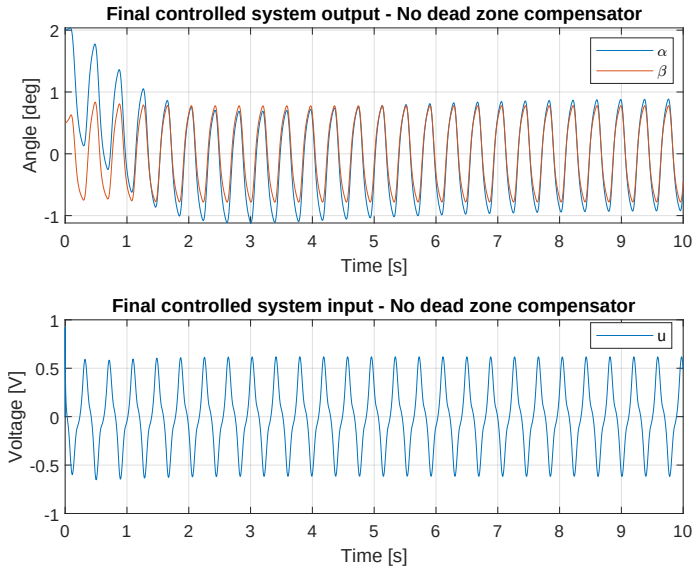


Figura 10: Controlled system feedback

A deadzone compensation algorithm was developed to mitigate these nonlinear effects. The control logic determines the polarity of the input voltage, u, and applies an instantaneous offset of $+0.20$ [V] or -0.20 [V], respectively. This effectively forces the control signal to bypass the deadzone region. This logic was realized via a `MATLAB Function` block and integrated into the primary Simulink diagram. The results are present in the following plot:

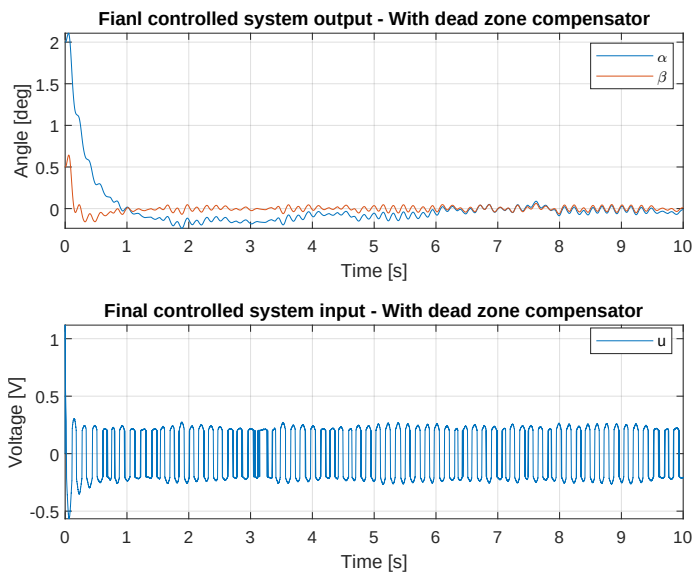


Figura 11: Controlled system feedback with deadzone compensator

Following the implementation of the compensator, further simulations were conducted. The results demonstrate that the angle signals exhibit a reduced amplitude and the characteristic "voltage jumping" is clearly visible in the input signal. Overall, the deadzone is effectively bypassed, leading to superior control authority and enhanced system performance.

IV. Conclusion

V. Anexos